

Architecture Reference

WBRS Performance Dashboard

System integrations, database structures, API dependencies,
and module roadmap for the operational back-office platform

Data Sources: 7shifts | Xenial POS | HME Cloud | Qualtrics | Google Places

Future Modules: Training | Inventory | User Provisioning | CRM | Payroll | Analytics

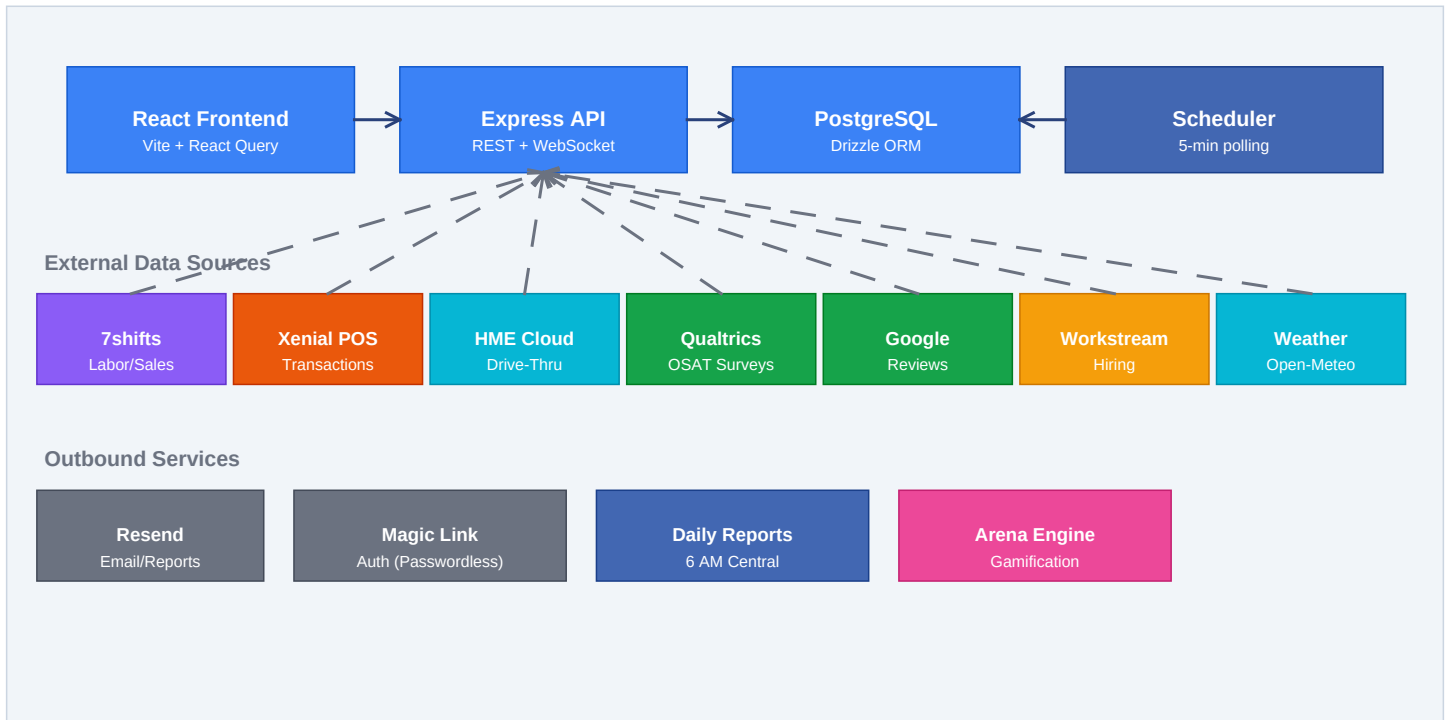
Table of Contents

1. System Architecture Overview
2. External Integration Map
3. Database Structure & Entity Relationships
4. Cross-System Identity Linking
5. 7shifts API Deep Dive
6. Sync Pipeline & Data Flow
7. Replacing 7shifts - Migration Playbook
8. Module Roadmap & Future Integrations
9. Environment Variables Reference

1. System Architecture Overview

The WBRS Performance Dashboard is a full-stack TypeScript monorepo that aggregates real-time data from multiple external sources into a unified restaurant performance platform. The backend (Express 5) syncs data every 5 minutes, stores it in PostgreSQL via Drizzle ORM, and serves normalized APIs. The frontend (React 18 + Vite) consumes these APIs through React Query.

High-Level Architecture Diagram



Technology Stack:

Backend: Express 5, TypeScript, Drizzle ORM, PostgreSQL, Passport (session auth)

Frontend: React 18, Vite, TanStack React Query, Recharts, Radix UI, Tailwind CSS

Hosting: Replit | Email: Resend | Auth: Passwordless magic-link

2. External Integration Map

Each external system connects via a specific method (REST API polling, webhook, or file export). All data is normalized into the internal PostgreSQL schema before the frontend consumes it.

Integration	Role	Sync Method	Frequency	Auth Method
7shifts	Labor mgmt, sales forecasts	REST API v2 (poll)	Every 5 min	Bearer token
Xenial POS	Real-time transactions	Inbound webhook	Real-time	Open (no auth)
HME Cloud	Drive-thru timers	REST API (poll)	Every 5 min	Service acct + key
Qualtrics	Customer satisfaction	IDP export + CSV	Every 5 min	API token
Google Places	Review ratings	REST API (poll)	Hourly	API key
Workstream	Hiring pipeline	REST API	On-demand	API token
Open-Meteo	Weather data	REST API	Hourly + EOD	None (public)
Resend	Transactional email	REST API	On-demand	API key

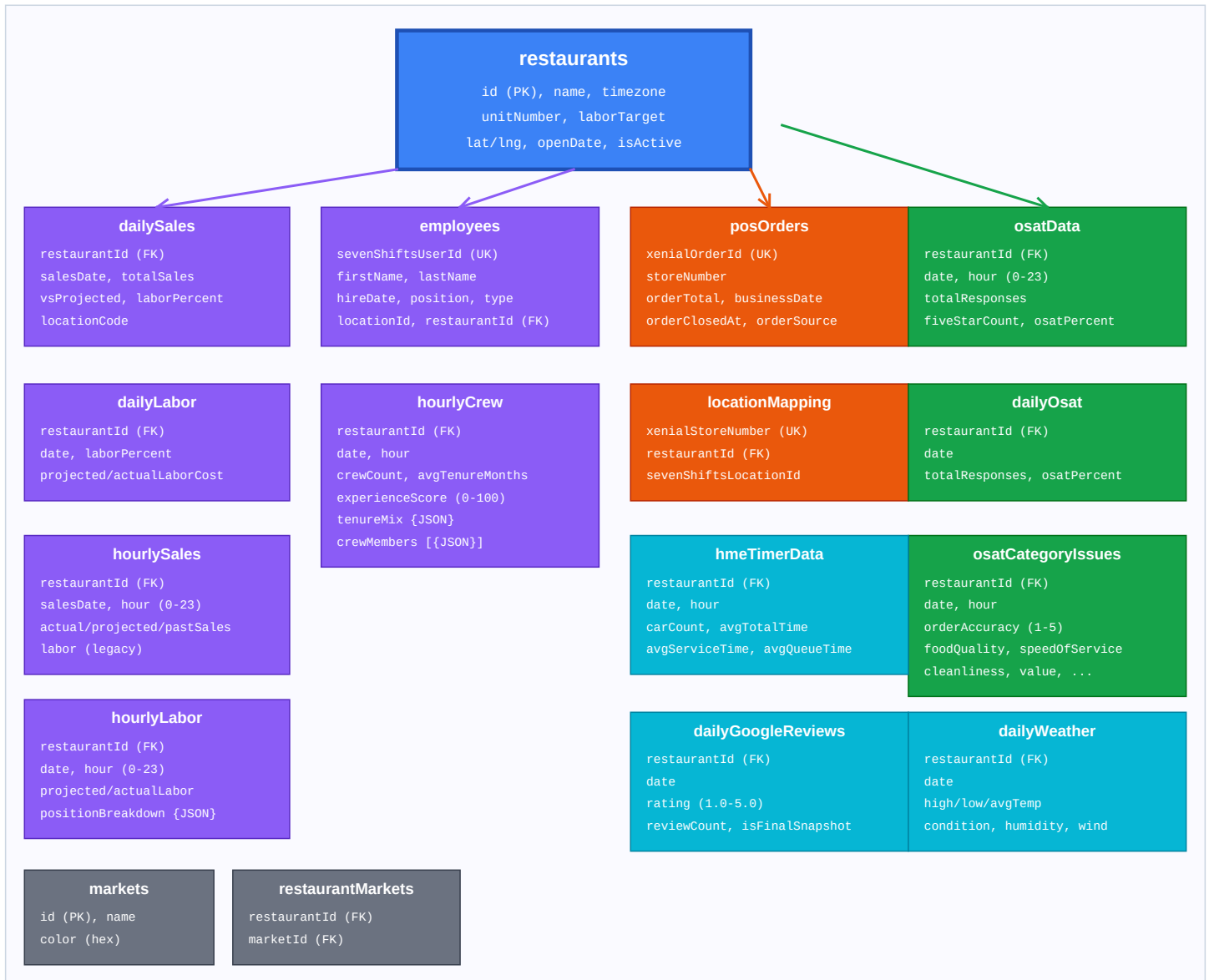
Data Volume & Priority

Source	Rows/Day (est.)	Tables Written	Business Priority
7shifts	~400 hourly + 22 daily	dailySales, dailyLabor, hourlySales, hourlyLabor, hourlyCrew, empl	CRITICAL
Xenial POS	~5,000-10,000 orders	posOrders (overlaid on hourlySales)	CRITICAL
HME Cloud	~400 hourly	hmeTimerData	HIGH
Qualtrics	~100-300 responses/day	osatData, dailyOsat, osatCategoryIssues	HIGH
Google Places	22 daily	dailyGoogleReviews	MEDIUM
Weather	22 daily	dailyWeather	LOW

3. Database Structure & Entity Relationships

All tables use UUID primary keys (gen_random_uuid). The restaurants table is the central entity - every data table references it via restaurantId. Data is organized by domain: labor (from 7shifts), sales (7shifts + Xenial), quality (Qualtrics + Google), operations (HME + Weather), and platform (auth, config, gamification).

Entity Relationship Diagram



Legend:

■ Core	■ Labor / 7shifts	■ POS / Xenial	■ Quality / OSAT
■ Enrichment	■ Arena / Gamification	■ Hiring / Workstream	■ Auth / Config

Arena (Gamification) & Platform Tables

arenaBadgesEarned badgeId, entityId entityType (leader/unit) earnedAt, metricValue shiftTeamMembers [{JSON}]	arenaStreaks entityId, entityType streakStart, streakCount streakActive, endedAt	arenaRecords recordType, holderId holderType, value evalDate, teamMembers	arenaMessages recipientEmail, messageType subject, message auto (bool), team (bool)
users username (UK), password email, role, displayName	magicLinkTokens email, tokenHash expiresAt, consumedAt	emailSubscribers email (UK), name reportTime, reportTypes[]	applicants digestKey (UK) restaurantId (FK) positionTitle, status appliedAt, hiredAt

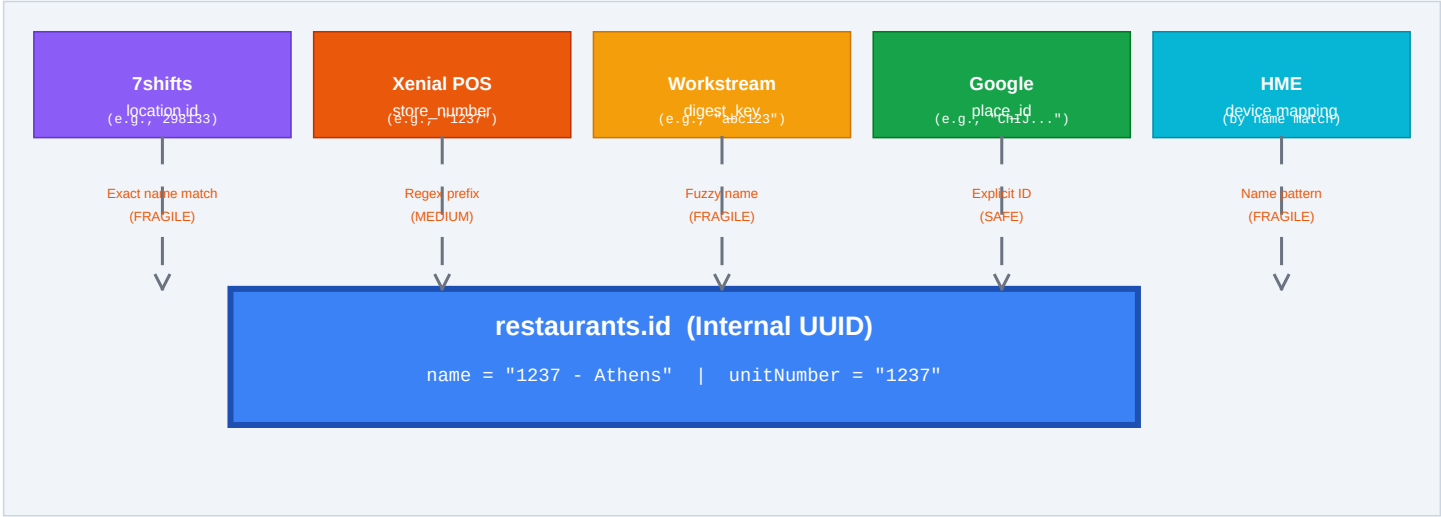
Complete Table Reference

Table	Source System	Grain	Key Columns
restaurants	7shifts (sync)	1 row/location	id, name, timezone, unitNumber, laborTarget
dailySales	7shifts + Xenial	1 row/day/restaurant	restaurantId, salesDate, totalSales, vsProjected
dailyLabor	7shifts	1 row/day/restaurant	restaurantId, date, laborPercent, actual/projectedCost
hourlySales	7shifts + Xenial	1 row/hour/restaurant	restaurantId, salesDate, hour, actual/projected
hourlyLabor	7shifts punches	1 row/hour/restaurant	restaurantId, date, hour, positionBreakdown
hourlyCrew	7shifts derived	1 row/hour/restaurant	restaurantId, date, hour, tenureMix, crewMembers
employees	7shifts	1 row/employee	sevenShiftsUserId, name, hireDate, position
posOrders	Xenial webhook	1 row/transaction	xenialOrderId, storeNumber, orderTotal
locationMapping	Hardcoded	1 row/store	xenialStoreNumber, restaurantId, sevenShiftsLocationId
hmeTimerData	HME Cloud	1 row/hour/restaurant	restaurantId, date, hour, carCount, avgServiceTime
osatData	Qualtrics	1 row/hour/restaurant	restaurantId, date, hour, osatPercent
dailyOsat	Qualtrics	1 row/day/restaurant	restaurantId, date, osatPercent
osatCategoryIssues	Qualtrics	1 row/response	restaurantId, date, hour, category ratings 1-5
dailyGoogleReviews	Google Places	1 row/day/restaurant	restaurantId, date, rating, reviewCount
dailyWeather	Open-Meteo	1 row/day/restaurant	restaurantId, date, temp, condition
applicants	Workstream	1 row/applicant	digestKey, restaurantId, positionTitle, status

4. Cross-System Identity Linking

Every external system has its own location identifier. These are currently linked through name-matching patterns and a hardcoded mapping table. This is the most critical architecture concern for adding or replacing integrations.

Location Identity Map



Current Matching Strategies & Risk Assessment:

From System	To System	Method	Risk Level	Failure Mode
7shifts	restaurants	Exact name match via location.name	HIGH	Renaming location breaks sync
Xenial POS	restaurants	Regex: name.match(/^(\d{4})\s*-/)	MEDIUM	Non-standard name format breaks
Xenial	7shifts	Hardcoded array (xenial-webhook.ts:340)	VERY HIGH	New locations need code deploy
Workstream	restaurants	Fuzzy name match during sync	HIGH	Name differences cause orphans
Google Places	restaurants	googlePlaceId column on restaurants	LOW	Explicit ID - safe
HME	restaurants	Name pattern matching	HIGH	Name inconsistency causes gaps

Recommended Future State:

Move all external IDs to explicit columns on the restaurants table. This eliminates name-matching fragility and makes adding new integrations straightforward. The restaurants table should become a canonical location registry with one column per external system ID.

5. 7shifts API Deep Dive

7shifts is the PRIMARY data backbone of the system. It provides location data, sales forecasts, labor cost/hours, employee records, time punches, role definitions, and scheduled shifts. The API client is in server/scrapper/7shifts-api.ts (~1,960 lines).

API Endpoints Used

Endpoint	Data Returned	Stored In	Frequency
GET /v2/whoami	Company ID, auth validation	In-memory	On startup
GET /v2/company/{id}/locations	All locations: name, tz, lat/lng	restaurants	On startup
GET /v2/reports/daily_sales_and_labor	Daily: actual/projected sales, labor %	dailySales, dailyLabor	Every 5 min
GET /v2/.../daily_stats	Hourly: sales intervals, labor	hourlySales, hourlyLabor	Every 5 min
GET /v2/company/{id}/time_punches	Clock in/out, role_id, breaks	Derived -> hourlyLabor	Every 5 min
GET /v2/company/{id}/roles	role_id -> position name mapping	In-memory (runtime)	Per sync
GET /v2/company/{id}/shifts	Scheduled shifts with roles	Operator detection	Per sync
GET /v2/company/{id}/users	Employee name, hire_date, type	employees	Daily

7shifts-Specific Concepts Embedded in the System

Employee Tenure Categories (hardcoded thresholds):

Category	Tenure Threshold	Experience Score Weight	Used In
Trainee	< 3 months	25 points	hourlyCrew.tenureMix, experience score
Developing	3-6 months	50 points	hourlyCrew.tenureMix, experience score
Experienced	6-12 months	75 points	hourlyCrew.tenureMix, experience score
Veteran	12+ months	100 points	hourlyCrew.tenureMix, experience score

Data Format Conventions:

- Sales values from 7shifts are in INTEGER CENTS (divided by 100 before storage)
- labor_percent is returned as decimal 0-1 (multiplied by 100 before storage)
- projected_labor_cost is in cents (divided by 100)
- Time punches use ISO timestamps with timezone offsets
- Fractional labor hours: a punch 10:30-11:30 = 0.5 hrs in hour 10 + 0.5 hrs in hour 11
- Operator detection: checks if scheduled shift role name includes 'Operator'
- Overnight handling: 4-hour padding before/after business day for spanning shifts

Files with Direct 7shifts Coupling

File	~Lines	Coupling Type
server/scrapper/7shifts-api.ts	1,960	Core API client - all sync functions
server/scrapper/7shifts-scrapers.ts	391	Browser scraper fallback (Playwright)
server/scheduler.ts	546	Orchestrates all 7shifts sync calls
server/xenial-webhook.ts	407	Hardcoded 7shifts location ID mappings
shared/schema.ts	710	sevenShiftsUserId, sevenShiftsLocationId columns
server/storage.ts	450+	Labor data lookups keyed by 7shifts-populated data

6. Sync Pipeline & Data Flow

The scheduler runs every 5 minutes, executing sync tasks in a specific order. Each task is independent but executed sequentially to avoid overwhelming external APIs.

Scheduler Execution Flow (every 5 minutes)

1	<code>fetchSalesFromAPI()</code>	7shifts daily sales + labor %
2	<code>fetchHourlySalesFromAPI()</code>	7shifts hourly intervals + time punches
3	<code>syncSalesWithXenialPOS()</code>	Overlay Xenial POS real-time sales
4	<code>syncHMETimerData()</code>	HME drive-thru car counts + times
5	<code>syncCrewExperience...()</code>	Hourly crew (top of hour only)
6	<code>syncGoogleReviews...()</code>	Google review ratings (hourly)
7	<code>syncOsatIfNeeded()</code>	Qualtrics OSAT surveys
8	<code>saveEndOfDayWeather...()</code>	Weather snapshot (11 PM CT only)
9	<code>syncYesterday...()</code>	Re-sync yesterday (midnight-6AM CT)
10	<code>sendDailyReports...()</code>	Email reports at configured time

Data Layering Strategy

The system uses a layered approach where 7shifts provides the baseline and other sources overlay or enrich the same time slots:

Layer 1	7shifts baseline	Sales forecasts, projected labor, hourly intervals
Layer 2	Xenial POS overlay	Replaces 7shifts sales with real-time POS totals
Layer 3	7shifts time punches	Actual labor hours by position (separate table)
Layer 4	HME enrichment	Drive-thru metrics attached to same hourly slots
Layer 5	Qualtrics enrichment	OSAT scores attached to same hourly slots
Layer 6	Google Places	Daily review rating snapshots
Layer 7	Weather	Daily weather context per location

Startup Sequence

1. Check if 7-day historical data exists (needed for week-over-week comparisons)
2. If missing: PAUSE scheduler -> backfill 9 days of daily + hourly data -> RESUME
3. Start scheduler timer (aligned to 5-min intervals)
4. Run initial sync immediately (ensures data on fresh deploy)
5. Force crew sync for today + yesterday
6. Schedule daily data cleanup (midnight, 730-day retention)

7. Replacing 7shifts - Migration Playbook

If 7shifts is replaced with another labor management system, the migration touches 6+ files and requires careful handling of identity, data formats, and historical continuity.

What a Replacement System Must Provide

Capability	7shifts Source	Required From Replacement	Critical?
Location list	/v2/.../locations	All locations with name, tz, lat/long	YES
Daily sales summary	/v2/reports/daily_sales_and_labor	Actual/projected sales, labor %, cost	YES
Hourly intervals	/v2/.../daily_stats	Per-hour actual/projected/past sales + labor	YES
Time punches	/v2/.../time_punches	Per-employee clock in/out with role + location	YES
Role/position map	/v2/.../roles	role_id -> human-readable position name	YES
Employee directory	/v2/.../users	Name, hire_date, active status, type, location	YES
Scheduled shifts	/v2/.../shifts	Start/end with role (for Operator detection)	MEDIUM
Sales forecasting	Embedded in daily_stats	Projected sales by hour and day	MEDIUM

4-Phase Migration Plan

Phase 1: Create Provider Abstraction

Create `server/providers/labor-provider.ts` with a common interface (`getLocations`, `getDailySummary`, `getHourlyIntervals`, `getTimePunches`, `getRoles`, `getEmployees`, `getScheduledShifts`). Refactor current 7shifts code to implement this interface.

Phase 2: Normalize Identity

Replace name-based matching with explicit ID columns on the restaurants table. Add `laborPlatformLocationId` column. Populate from current 7shifts data. Update sync functions to use explicit ID matching.

Phase 3: Decouple Schema

Rename 7shifts-specific columns: `employees.sevenShiftsUserId` -> `externalUserId` (+ `externalSource` field), `locationMapping.sevenShiftsLocationId` -> `laborPlatformLocationId`. Add migration scripts for column renames.

Phase 4: Swap API Client

Implement new provider (e.g., `HotSchedules`, `R365`, etc.). Update `scheduler.ts` imports. Map new data formats to normalized structures. Run parallel (old + new) before cutover.

Hardcoded Values Requiring Update

Value	Location	Action Required
<code>https://api.7shifts.com</code>	<code>7shifts-api.ts:139</code>	Replace with new provider base URL
<code>SEVENSHIFTS_API_TOKEN</code> env var	Multiple files	Replace with new provider auth
Store <-> 7shifts ID array	<code>xenial-webhook.ts:340-362</code>	Update with new labor system IDs
Employee type values	<code>7shifts-api.ts</code> (employee, manager...)	Map new provider's role types
/v2/ endpoint paths	Throughout <code>7shifts-api.ts</code>	Replace with new API paths
Cents-to-dollars conversion	<code>7shifts-api.ts</code> ($\div 100$)	Verify new provider's units

Data Continuity Notes:

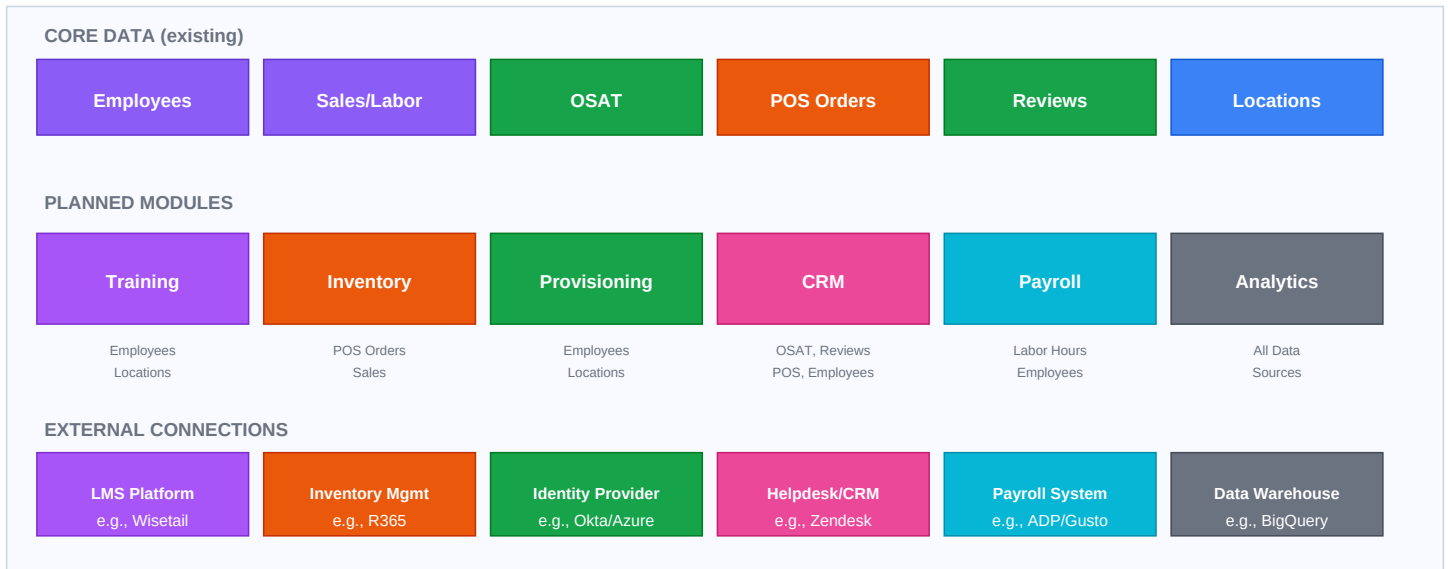
- Historical data is safe: hourly/daily data tables have no 7shifts-specific format

- Employee records need mapping: sevenShiftsUserId is the unique key; new provider has different IDs
- Tenure calculations continue if new system provides hire_date
- Position names may differ: '7shifts Grill' vs new system's equivalent. Update position logic.
- The frontend is already decoupled: it consumes normalized TypeScript interfaces, NOT provider data

8. Module Roadmap & Future Integrations

The following modules are planned to extend the operational back-office platform. Each module builds on existing data and the restaurants/employees entity graph.

Integration Dependency Map



8.1 Training System

Purpose: Training tracking, certification management, onboarding workflows.

Data Dependencies:

- employees table (name, position, hireDate, restaurantId) for enrollment triggers
- Tenure categories (Trainee < 3mo) as auto-enrollment triggers
- hourlyCrew.crewMembers for on-shift training status

New Tables: trainingCourses, trainingEnrollments, trainingCertifications

Trigger: syncEmployees() creates new employee -> auto-enroll in position-appropriate courses

8.2 Inventory & Ordering

Purpose: Track inventory levels, COGS, waste, and product-level sales mix.

Data Dependencies:

- posOrders (Xenial webhook) for revenue data. NOTE: product-level line items are NOT currently parsed
- posOrders.rawJson MAY contain line-item data that can be parsed
- hourlySales for revenue context

New Tables: products, inventoryLevels, orderItems, purchaseOrders

Action: Parse line items from existing rawJson OR add Xenial webhook entity type

8.3 User Provisioning & Lifecycle

Purpose: Auto-provision/deprovision accounts across systems on hire/transfer/termination.

Data Dependencies:

- Employee sync detects new hires (inserts) and status changes (active flag)
- applicants table tracks 'hired' status from Workstream
- employees.locationId changes indicate transfers

New Tables: provisioningEvents, provisioningRules

Events: employee.created -> invite to POS + training + comms

employee.locationChanged -> update access across systems

employee.deactivated -> revoke all system access

8.4 CRM (Customer Relationship Management)

Purpose: Support case management, customer interaction tracking, marketing campaigns.

Data Dependencies:

- osatData / osatCategoryIssues: low scores auto-create support cases
- dailyGoogleReviews: rating drops trigger review alerts
- posOrders: link to customer purchase history
- restaurants: geographic context for regional marketing

New Tables: customers, customerInteractions, supportCases, marketingCampaigns

Auto-triggers:

- OSAT < 60% -> create 'OSAT Alert' case
- Google rating drop > 0.2 in a day -> create 'Review Alert' case
- New 1-star survey -> create individual follow-up case

8.5 Payroll Integration

Purpose: Export labor hours, overtime, and position data to payroll processor.

Data Dependencies:

- hourlyLabor: position breakdowns and hours per employee
- dailyLabor: daily totals and labor cost
- employees: names, positions, locations, hire dates

New Tables: payrollExports, payrollLineItems

Export: Aggregate hourlyLabor into weekly summaries per employee,
map positions to pay rates, export CSV/API to ADP, Gusto, Paychex, etc.

8.6 Analytics & Financial Export

Purpose: Export operational data to financial systems, BI tools, accounting platforms.

Data Dependencies:

- ALL data tables are potential export sources
- dailySales + dailyLabor are primary financial metrics
- posOrders for transaction-level detail

New Tables: exportConfigurations, exportRuns

Destinations: QuickBooks, Xero, NetSuite, Looker, Tableau, BigQuery, Snowflake

Formats: CSV (accounting), JSON API (push), SQL (replication), Webhook (event-driven)

9. Environment Variables Reference

Required Variables

Variable	Used By	Purpose
DATABASE_URL	server/db.ts	Primary PostgreSQL connection string
SEVENSHIFTS_API_TOKEN	server/scrapper/7shifts-api.ts	7shifts API bearer token
HME_SERVICE_ACCOUNT	server/scrapper/hme-api.ts	HME Cloud service account ID
HME_AUTH_KEY	server/scrapper/hme-api.ts	HME Cloud authentication key
HME_ACCOUNT_EMAIL	server/scrapper/hme-api.ts	HME Cloud account email
QUALTRICS_API_TOKEN	server/scrapper/qualtrics-api.ts	Qualtrics API authentication token
QUALTRICS_IDP_SOURCE_ID	server/scrapper/qualtrics-api.ts	Qualtrics imported data project ID
GOOGLE_PLACES_API_KEY	server/google-places.ts	Google Places API key
RESEND_API_KEY	server/email.ts	Resend email service API key
ALLOWED_LOGIN_EMAILS	server/routes/auth.ts	Comma-separated email whitelist
SESSION_SECRET	server/index.ts	Express session encryption key

Optional Variables

Variable	Used By	Purpose
XPOSSHARED_DATABASE_URL	server/db.ts	Separate POS database (fallback: DATABASE_URL)
SHARED_DATABASE_URL	server/db.ts	Fallback for both database URLs
RESEND_FROM_EMAIL	server/email.ts	Custom from address for emails
WORKSTREAM_API_TOKEN	server/scrapper/7shifts-scraper.ts	Workstream hiring platform auth
GOOGLE_PLACES_LOCATION_CACHE	server/google-places.ts	Cached place IDs (JSON)
SEVENSHIFTS_EMAIL	server/scrapper/7shifts-scraper.ts	Browser scraper login email
SEVENSHIFTS_PASSWORD	server/scrapper/7shifts-scraper.ts	Browser scraper login password

Key Takeaways

- 1 Frontend is already decoupled**

The React frontend consumes normalized TypeScript interfaces (RestaurantSales, HourlySalesData, etc.). Any backend provider swap is invisible to the frontend. No frontend changes required.
- 2 Location identity is the biggest migration risk**

The current system relies on name matching ('1237 - Athens'). Move to explicit ID columns on the restaurants table before adding more integrations. This is the single highest-priority refactor.
- 3 Employee identity is tied to sevenShiftsUserId**

Any new labor platform introduces a new ID space. Plan for a mapping/merge strategy with a transitional externalUserId + externalSource column approach.
- 4 The scheduler is the integration orchestrator**

All new data sources should follow the same pattern: create a sync function, add it to the scheduler with appropriate frequency and deduplication guards (like the syncKey pattern).
- 5 The data layering strategy works well**

7shifts provides baselines, Xenial POS overlays real-time sales, enrichment sources attach to the same hourly time slots. New modules should follow this pattern of additive layers.
- 6 Hardcoded mappings are tech debt priority #1**

The Xenial-to-7shifts mapping array in xenial-webhook.ts:340-362 and name-matching patterns throughout will not scale. Every new location currently requires a code deployment.

This document should be updated as integrations are added, replaced, or modified.

Source: docs/architecture-reference.md | Generated: February 2026